

Key factors that need to be kept in mind while delivering as fast as possible is the software's high dependency on previous principles like amplified learning and deciding as late as possible, and focused productive work which can be hard to achieve if the team doesn't have the synergy or the motivation. To summarise, one needs to have the right team while keeping it simple to deliver the product fast.

3. Building quality/integrity

For an open source product manager or developer, quality is crucial. It is very important to set a clear quality standard at the beginning of any project, which in turn streamlines the quality, integrity and maintainability of the project. Building and maintaining the quality of a product starts at the very foundation of a project with the definition of *Done*.

Done is a set of rules and practices defined at the start of any project by the developers, and approved by the stakeholders and the product manager, of how each module of the project should perform. This definition can be extensive and general or short, according to the level of quality the project requires.

4. Deciding as late as possible

Deciding as late as possible means that final decisions for further directives should only be made when they are required. This helps us in keeping the options open for as long as possible and leads to the following benefits. First, it gives us more time to acquire knowledge and explore other potential options in depth. Second, it helps in knowing and learning more about which option to Choose, based on product feedback and other changes.

Deciding as late as possible optimises the upside reward by delaying decisions until more knowledge is secured. However, this is only one of the two major benefits of this principle. Another major benefit of deciding late is that it limits the downside risk by eliminating the cost and time allocated to resolving

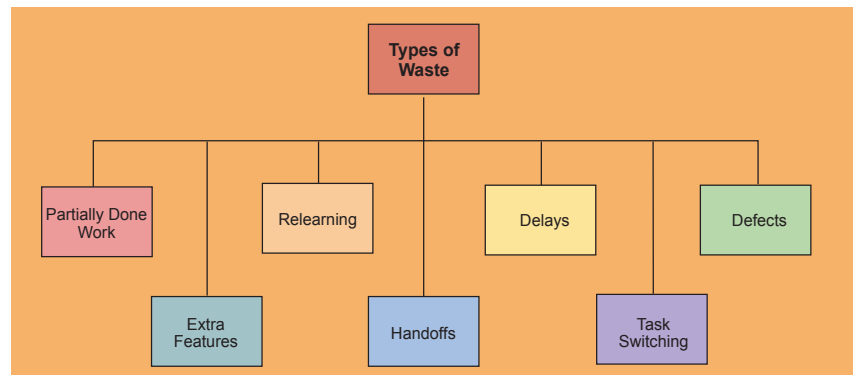


Figure 3: Types of waste

uncertainty. Let us understand this with the help of an example.

HP sells printers all over the world, but it couldn't accurately forecast how many printers it could sell in each country. There were always some excess printers in one country and not enough for another country. Moreover, they couldn't sell printers configured for one country in another because the electrical configurations were different for each country. Then the company decided to do the final electrical configuration in the warehouses *after* the printer was ordered. Even though costs rose for a single unit, HP saved US\$ 3 million a month by more effectively matching supply to demand. This is a classic example of deciding as late as possible to limit downside risk.

5. Optimising the whole

'Optimising the whole' encourages open source software developers to eliminate vicious cycles by operating with a better understanding of their own capacity. What is a vicious cycle, one might ask. A major vicious cycle that leads to sub-optimisation works as follows:

- Users demand some new features/bug solutions urgently.
- Developers are under pressure to get the changes ready instantly and at all costs.
- This results in sloppy changes to the code in the developers' respective modules.

- Complexity of the code base increases.
- Complexity of integration increases exponentially.
- Number of defects in the code increases.

To get rid of this vicious cycle, we need to zoom out and look at the bigger picture. If we do so, we soon realise that most big delays, communications issues and misunderstandings are caused when the work of one developer is handed off to another developer. Thus, it is extremely important to focus on integration, one of the wastes explained in 'eliminate waste'. Moreover, using kanban for workflow visualisation, measuring flow, prioritising, and managing flow goes a long way in optimising the whole process. Other than that, open source developers should organise teams by modules rather than by skills. This may help not only in optimising the workflows and integrating handoffs, but also by letting developers have a better sense of ownership for the modules they are working on.

6. Empowering the team

Timid teams and new developers can often wait for a long time for trivial decisions from more experienced developers. This can introduce a lot of delays. A good team leader should always extend some decision making freedom. However, the extent to which decisions can be made by new developers without the approval of experienced developers/mentors/managers should be clarified